

Cómo aplicar el SSL a tu aplicativo.

1. Instalar SSL con el siguiente comando, en el CMD.

```
winget install -e --id ShiningLight.OpenSSL
```

1.1. Si tienes un error que dice: “No se encontró ningún paquete coincidente con los criterios de búsqueda. “ Intenta con el siguiente comando, dentro de la carpeta sobre la que quieres instalar el ssl:

```
winget install openssl
```

2. Verificar la versión del SSL con el siguiente comando:
pdta. Tiene que ser una versión mayor de 1.1.1, en determinado caso que necesites actualizar puedes utilizar este comando: “npm update openssl”

```
openssl version
```

3. Una vez ubicado en la carpeta en la cual tienes tu proyecto, generar la llave del localhost con el siguiente comando:

```
openssl genrsa -out localhost.key 2048
```

Después de esto, la consola no tendrá que darte ningún resultado, lo cual es una excelente señal.

4. Después del paso anterior generando la llave para el localhost, necesitamos generar un certificado autofirmado, se genera principalmente para pruebas y para dar un buen resultado del avance y desarrollo del aplicativo.

```
openssl req -x509 -new -nodes -key localhost.key -sha256 -days 365 -out localhost.crt -subj "/C=CO/ST=Bogota/L=Bogota/O=SENA/CN=localhost"
```

Lo que está subrayado son las cosas a las que tu le tienes que dar un poquito más de atención, pues en esta tenemos que registrar unos valores para que la credencial sea un

poco más informativa, como cuantos días va a durar esta certificación, y donde está ubicada, con país, departamento, ciudad, y organización.

5. Una vez realizado el certificado tiene que pasar lo mismo con el punto 3, no tiene que aparecer nada en la terminal, y eso es una muy buena señal, ahora necesitamos validar el certificado, eso lo realizamos con el siguiente comando.

```
openssl x509 -in localhost.crt -text -noout
```

Te generará 2 archivos nuevos en tu proyecto con los .key y .crt, estos archivos son muy importantes que los ubiques en una carpeta llamada "certs". Esta carpeta tiene que ir en las carpetas tanto en Frontend como en la del Backend, si no lo haces de la forma correcta te aparecerá un error en la terminal al momento de ejecutar como este:

```
shurt@PcSanti MINGW64 ~/OneDrive/Escritorio/Ahorrrapp-REACT/Frontend (Santiago)
$ npm run dev

> ahorrrapp@0.0.0 dev
> vite

failed to load config from C:\Users\shurt\OneDrive\Escritorio\Ahorrrapp-REACT\Frontend\vite.config.js
error when starting dev server:
Error: ENOENT: no such file or directory, open 'C:\Users\shurt\OneDrive\Escritorio\Ahorrrapp-REACT\Frontend\certs\localhost.key'
    at Object.openSync (node:fs:560:18)
    at Object.readFileSync (node:fs:444:35)
    at file:///C:/Users/shurt/OneDrive/Escritorio/Ahorrrapp-REACT/Frontend/node_modules/.vite-temp/vite.config.js.timestamp-1780458684166-707588614de2e8.mjs:11:15
    at ModuleJob.run (node:internal/modules/esm/module_job:430:25)
    at async onImport.tracePromise.__proto__ (node:internal/modules/esm/loader:661:26)
    at async loadConfigFromBundledFile (file:///C:/Users/shurt/OneDrive/Escritorio/Ahorrrapp-REACT/Frontend/node_modules/vite/dist/node/chunks/config.js:35936:12)
    at async bundleAndLoadConfigFile (file:///C:/Users/shurt/OneDrive/Escritorio/Ahorrrapp-REACT/Frontend/node_modules/vite/dist/node/chunks/config.js:35824:17)
    at async loadConfigFromFile (file:///C:/Users/shurt/OneDrive/Escritorio/Ahorrrapp-REACT/Frontend/node_modules/vite/dist/node/chunks/config.js:35791:42)
    at async resolveConfig (file:///C:/Users/shurt/OneDrive/Escritorio/Ahorrrapp-REACT/Frontend/node_modules/vite/dist/node/chunks/config.js:35440:22)
    at async _createServer (file:///C:/Users/shurt/OneDrive/Escritorio/Ahorrrapp-REACT/Frontend/node_modules/vite/dist/node/chunks/config.js:25381:67)
```

6. Después de esto, tienes que crear 2 archivos para que esté https, funcione correctamente, ¿Cuáles son? Estarán ubicados en el backend, y serán el [server.js](#) y el [app.js](#), estos serán los responsables de generar la configuración para el servidor con https, estos tienen que tener una estructura como esta:

[api.js](#)

```
const express = require('express');
const app = express();

app.use(express.json());

app.get('/', (req, res) => {
  res.json({ ok: true, mensaje: 'Servidor AhorrApp corriendo' });
});

app.get('/api', (req, res) => {
  res.json({ ok: true, mensaje: 'API AhorrApp corriendo' });
});

module.exports = app;
```

[server.js](#)

```
const https = require('https');
const http = require('http');
const fs = require('fs');
const app = require('./app');

const opciones = {
  key: fs.readFileSync('certs/localhost.key'),
  cert: fs.readFileSync('certs/localhost.crt'),
};

https.createServer(opciones, app).listen(443, () => {
  console.log('✅ HTTPS Server running on port 443');
});

http.createServer((req, res) => {
  res.writeHead(301,
    { Location: 'https://' + req.headers.host + req.url });
  res.end();
}).listen(80, () => {
  console.log('🔗 HTTP Server running on port 80');
});
```

Básicamente, en el servidor se reflejan los archivos que creamos anteriormente.

7. Después ya sigue el paso final que es ejecutar el servidor y las carpetas de frontend y backend, el servidor lo ejecutas con el siguiente código:

```
node server.js
```

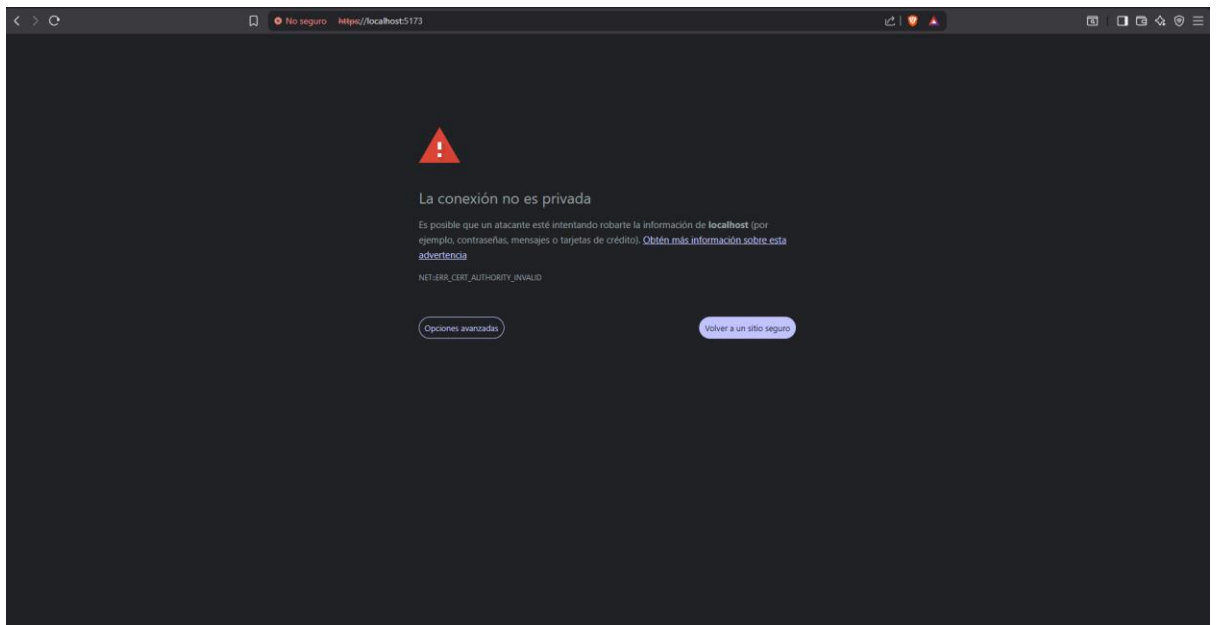
Después de esto, en la terminal se verá el console.log encargado de mostrar en la terminal si el servidor está ejecutando o no.

8. Ahora depende de donde hayas realizado tu proyecto tendrás que modificar el [vite.config.js](#) en este caso el mio es en react por lo cual tengo la siguiente configuración:

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import fs from 'fs';

export default defineConfig({
  plugins: [react()],
  server: {
    host: 'localhost',
    port: 5173,
    https: {
      key: fs.readFileSync('certs/localhost.key'),
      cert: fs.readFileSync('certs/localhost.crt'),
    },
  },
});
```

Una vez ya hayas realizado todo tendrás que ejecutar el servidor y las 2 carpetas. Acto seguido tendrás que ver en el navegador una vista tal que así



Esta es la señal que nos indica que funciona, ahora ¿Cómo entras a tu proyecto?
Presionamos en opciones avanzadas, y presionas “Continuar a localhost.”

Cambios de ubicación y de tiempo en las certificaciones de SSL..

Duración 60 días, ubicación Boyacá, Duitama, Organización Seguros del Estado.

```
C:\Users\shurt\OneDrive\Escritorio\Ahorrap-REACT\Frontend\certs>openssl x509 -in localhost.crt -text -noout
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      77:f4:81:ca:3c:1b:99:45:21:69:2e:5c:2b:a9:6a:a8:63:5e:fa:e7
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: C=CO, ST=Boyaca, L=Duitama, O=Seguros Del Estado, CN=localhost
    Validity
      Not Before: Jun  3 11:52:37 2026 GMT
      Not After : Aug  2 11:52:37 2026 GMT
    Subject: C=CO, ST=Boyaca, L=Duitama, O=Seguros Del Estado, CN=localhost
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      Public-Key: (2048 bit)
      Modulus:
        a6:d2:43:a9:28:1d:a6:89:a6:8b:86:a3:b8:c5:e4:a8:
        85:50:67:29:c1:96:87:c0:78:9a:76:e1:28:05:85:b1:
        f0:ae:eb:e8:ff:2c:02:2d:2a:99:a6:b1:aa:cd:cd:0e:
        07:47:37:30:41:3e:b2:ca:b7:15:0a:54:28:3b:a0:20:
        d6:d4:8c:94:59:10:78:ce:11:bb:6d:ac:8e:eb:24:12:
        16:85:45:85:79:a8:55:b0:57:a2:df:58:c2:67:8a:7b:
        2a:69:b4:d0:05:ae:c1:25:86:19:6a:92:7f:37:12:c4:
        46:bd:bb:15:95:de:dc:2f:a7:01:8b:88:0e:0c:cd:a3:
        12:c9:74:9f:2b:5a:ab:eb:fc:91:e1:d4:1a:a7:eb:45:
        4d:bf:4d:c6:c4:64:0d:b8:05:b2:6a:be:fb:a3:c7:50:
        a5:75:9a:01:ff:0f:41:ac:b0:7e:10:a4:5d:7e:4c:48:
        1e:14:29:d5:f0:8c:90:58:69:23:e6:7c:01:53:9a:f4:
        6b:2f:b0:03:2d:08:22:0b:ec:53:b0:49:ad:9e:55:f8:
        28:de:f4:91:7e:e6:15:a3:3f:8a:4b:d2:49:85:43:db:
        ee:69:f8:87:f6:a8:82:5a:1f:7c:cb:ce:fd:e9:be:66:
        75:f1:68:f3:07:b6:69:cd:e0:4b:92:ee:c6:31:ed:ed
      Exponent: 65537 (0x10001)
    X509v3 extensions:
      X509v3 Subject Key Identifier:
        FE:02:AE:9A:79:79:80:01:C6:BF:9E:A7:9B:0E:5F:3E:20:90:96:1F
      X509v3 Basic Constraints: critical
        CA:TRUE
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
      71:41:09:31:bb:93:46:bc:e5:d5:89:07:e3:b5:a2:fd:6e:04:14:b4:b2:23:73:fb:
      c0:07:e1:3b:da:56:14:11:a7:57:d6:f8:b7:de:b2:da:44:5a:ba:13:ab:48:da:0d:
      22:1c:b1:49:f5:bf:ea:0a:75:92:61:aa:be:a0:74:af:81:ee:05:e2:6a:c0:10:2d:
      b4:4d:53:3d:de:84:2d:e0:a5:2d:9d:30:31:e5:93:69:21:34:bb:63:7c:ea:86:23:
      c4:7d:cc:8f:ff:14:d6:c2:20:65:1b:bd:53:56:f2:99:95:0d:de:bf:86:d4:9d:b5:
      0b:2e:e1:dd:38:34:72:61:10:d4:fb:84:81:35:f5:3c:20:50:d8:50:ac:c8:03:ad:
      2d:f2:cf:d0:97:90:e0:f3:8a:e8:9b:3b:46:1b:2f:ea:97:ac:dc:8e:7f:19:44:c9:
      62:bc:18:b7:17:28:b0:8e:e3:00:e8:19:3f:ea:b4:70:92:d5:dc:ce:76:7e:de:56:
      3f:f0:9f:79:46:ef:0b:f8:27:7a:ca:ca:10:97:dd:01:0a:72:4e:04:a9:f5:0e:3b:
      4f:ee:97:c8:03:05:59:c5:fa:7e:ec:50:fc:ae:6a:54:34:29:e9:25:18:4c:f9:b3:
      8b:36:64:38:2f:0f:da:be:c7:af:a1:19:2d:f5:fc:5a
```

Duración 45 días, ubicación Antioquia, Medellín, Organizacion Ahorrapp.

```
C:\Users\shurt\OneDrive\Escritorio\Ahorrapp-REACT\Frontend\certs>openssl x509 -in localhost.crt -text -noout
Certificate:
```

```
  Data:
    Version: 3 (0x2)
    Serial Number:
      33:b9:e9:21:49:a8:5e:ba:9c:c9:3f:a2:32:d7:03:3a:a7:e0:9b:c7
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: C=CO, ST=Antioquia, L=Medellin, O=Ahorrapp, CN=localhost
    Validity
      Not Before: Jun  3 12:00:41 2026 GMT
      Not After : Jul 18 12:00:41 2026 GMT
    Subject: C=CO, ST=Antioquia, L=Medellin, O=Ahorrapp, CN=localhost
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      Public-Key: (2048 bit)
      Modulus:
        a6:d2:43:a9:28:1d:a6:89:a6:8b:86:a3:b8:c5:e4:a8:
        85:50:67:29:c1:96:87:c0:78:9a:76:e1:28:05:85:b1:
        f0:ae:eb:e8:ff:2c:02:2d:2a:99:a6:b1:aa:cd:cd:0e:
        07:47:37:30:41:3e:b2:ca:b7:15:0a:54:28:3b:a0:20:
        d6:d4:8c:94:59:10:78:ce:11:bb:6d:ac:8e:eb:24:12:
        16:85:45:85:79:a8:55:b0:57:a2:df:58:c2:67:8a:7b:
        2a:69:b4:d0:05:ae:c1:25:86:19:6a:92:7f:37:12:c4:
        46:bd:bb:15:95:de:dc:2f:a7:01:8b:88:0e:0c:cd:a3:
        12:c9:74:9f:2b:5a:ab:eb:fc:91:e1:d4:1a:a7:eb:45:
        4d:bf:4d:c6:c4:64:0d:b8:05:b2:6a:be:fb:a3:c7:50:
        a5:75:9a:01:ff:0f:41:ac:b0:7e:10:a4:5d:7e:4c:48:
        1e:14:29:d5:f0:8c:90:58:69:23:e6:7c:01:53:9a:f4:
        6b:2f:b0:03:2d:08:22:0b:ec:53:b0:49:ad:9e:55:f8:
        28:de:f4:91:7e:e6:15:a3:3f:8a:4b:d2:49:85:43:db:
        ee:69:f8:87:f6:a8:82:5a:1f:7c:cb:ce:fd:e9:be:66:
        75:f1:68:f3:07:b6:69:cd:e0:4b:92:ee:c6:31:ed:ed
      Exponent: 65537 (0x10001)
    X509v3 extensions:
      X509v3 Subject Key Identifier:
        FE:02:AE:9A:79:79:80:01:C6:BF:9E:A7:9B:0E:5F:3E:20:90:96:1F
      X509v3 Basic Constraints: critical
        CA:TRUE
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
      85:70:55:57:ed:5f:3f:ff:3e:9f:36:94:b3:a5:0f:99:23:3f:f0:4e:0f:bd:32:63:
      8b:37:8d:43:a6:80:dd:03:9a:7d:e7:4a:88:9e:03:fc:b3:2f:e5:5d:71:b1:08:ac:
      10:45:91:1c:2d:19:68:53:4f:e5:a1:7b:fd:fe:6d:92:39:7e:03:7f:47:a4:db:f7:
      1e:3f:41:96:42:5b:cc:b6:c3:24:7e:39:31:d5:9b:52:c6:7a:43:71:77:ed:23:de:
      49:a6:41:e4:6c:a5:5f:7c:c1:27:0d:41:54:7b:c0:71:2c:8b:01:81:c6:bd:29:f5:
      db:68:6f:f1:53:e5:72:e2:03:5d:97:e7:c7:27:09:56:45:0a:29:1a:6c:45:8b:22:
      ca:35:fc:02:c1:2a:3c:0e:4c:40:fd:cd:75:f8:9c:bd:73:06:da:87:37:0a:07:98:
      b8:94:af:38:c0:3f:2e:fc:37:d8:4b:9b:78:be:6f:2a:2d:b8:99:8c:b7:79:b4:d4:
      5d:c3:96:b6:b3:b4:b9:ed:51:25:e9:36:43:57:3a:05:9e:38:97:da:7e:3f:c6:76:
      ba:20:92:e5:81:0c:b6:4d:72:88:d0:ea:a4:21:43:d2:58:4b:25:5d:10:dd:8e:75:
      8a:f3:ca:c0:8c:44:b2:13:77:da:8c:1a:81:15:78:ae
```